

WEEK - 3

NAME: Nandhini V

DATE: 26-07-2025 & 27-07-2025

Question:

1. Create a class BankAccount in Python with private attributes `__accountno`, `__name`, `__balance`.

Add

parameterized constructor

methods:

`deposit(amount)`

`withdraw(amount)`

`set_accountno`

`get_accountno`

`set_name`

`get_name`

`get_balance()`

`set_balance()`

2. How will you define a static method in Python? Explore and give an example.

3. Give examples for dunder methods in Python other than `__str__` and `__init__`.

4. Explore some supervised and unsupervised models in ML.

5. Implement Stack with class in Python.

Answer:

1. Code

```
class BankAccount:
```

```
    def __init__(self, accountno, name, balance):
```

```
        self.__accountno = accountno
```

```
        self.__name = name
```

```
        self.__balance = balance
```

```
    def deposit(self, amount):
```

```
        if amount > 0:
```

```
            self.__balance += amount
```

```
            print(f'Deposited ₹{amount}. New balance: ₹{self.__balance}')
```

```
        else:
```

```
            print("Invalid deposit amount")
```

```
    def withdraw(self, amount):
```

```
        if 0 < amount <= self.__balance:
```

```
            self.__balance -= amount
```

```
            print(f'Withdrew ₹{amount}. New balance: ₹{self.__balance}')
```

```
        else:
```

```
            print("Invalid or insufficient balance for withdrawal")
```

```
    def set_accountno(self, accountno):
```

```
        self.__accountno = accountno
```

```
def get_accountno(self):  
    return self.__accountno
```

```
def set_name(self, name):  
    self.__name = name
```

```
def get_name(self):  
    return self.__name
```

```
def set_balance(self, balance):  
    if balance >= 0:  
        self.__balance = balance  
    else:  
        print("Balance cannot be negative")
```

```
def get_balance(self):  
    return self.__balance
```

```
account = BankAccount(1001, "Sri", 5000)
```

```
print("Account Number:", account.get_accountno())
```

```
print("Account Holder:", account.get_name())
```

```
print("Initial Balance:", account.get_balance())
```

```
account.deposit(2000)
```

```
account.withdraw(1500)
```

```
account.set_name("Srinidhi")
```

```
print("Updated Name:", account.get_name())
```

```
account.set_accountno(2002)
```

```
print("Updated Account Number:", account.get_accountno())
```

```
account.set_balance(10000)
```

```
print("Updated Balance:", account.get_balance())
```

Output:

```
PS D:\Task_Python> python bankacc.py
```

```
Account Number: 1001
```

```
Account Holder: Sri
```

```
Initial Balance: 5000
```

```
Deposited ₹2000. New balance: ₹7000
```

```
Withdrew ₹1500. New balance: ₹5500
```

```
Updated Account Number: 2002
```

```
Updated Balance: 10000
```

2. Static method

- A static method is defined using the `@staticmethod` decorator.
- It doesn't use `self` or `cls`, meaning it doesn't access instance or class variables.
- It's used for utility functions related to the class but not dependent on it.

Example:

```
class Utility:

    @staticmethod

    def greet(name):

        return f"Hello, {name}!"

print(Utility.greet("Navya"))
```

3. Dunder methods examples

- `__len__()` – returns length
- `__add__()` – defines + behavior
- `__eq__()` – defines equality ==
- `__getitem__()` – allows indexing

4. ML Models

Supervised:

- Linear Regression
- Decision Tree
- Support Vector Machine (SVM)

Unsupervised:

- K-Means Clustering
- PCA (Principal Component Analysis)
- DBSCAN

5. Code

```
class Stack:
```

```
    def __init__(self):
```

```
        self.stack = []
```

```
    def push(self, item):
```

```
        self.stack.append(item)
```

```
        print(f"Pushed {item} to stack")
```

```
    def pop(self):
```

```
        if not self.is_empty():
```

```
            item = self.stack.pop()
```

```
            print(f"Popped {item} from stack")
```

```
            return item
```

```
        else:
```

```
            print("Stack is empty")
```

```
            return None
```

```
    def peek(self):
```

```
        if not self.is_empty():
```

```
            return self.stack[-1]
```

```
        else:
```

```
            print("Stack is empty")
```

```
            return None
```

```
def is_empty(self):  
    return len(self.stack) == 0  
  
def display(self):  
    print("Stack contents:", self.stack)
```

```
s = Stack()  
s.push(10)  
s.push(20)  
s.push(30)  
s.display()  
print("Top of stack:", s.peek())  
s.pop()  
s.display()  
print("Is stack empty?", s.is_empty())
```

Output:

```
PS D:\Task_Python> python stackclass.py
```

```
Pushed 10 to stack
```

```
Pushed 20 to stack
```

```
Pushed 30 to stack
```

```
Stack contents: [10, 20, 30]
```

```
Top of stack: 30
```

```
Popped 30 from stack
```

```
Stack contents: [10, 20]
```

```
Is stack empty? False
```